

```
# 1. Mise à jour de protobuf pour corriger le conflit de version
!pip install --upgrade protobuf tensorflow-metadata

# 2. Redémarrage automatique du notebook pour appliquer les changements
import os
os.kill(os.getpid(), 9)
```

```
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (5.29.6)
Collecting protobuf
  Downloading protobuf-7.35.1-cp310-abi3-manylinux2014_x86_64.whl.metadata (595 bytes)
Requirement already satisfied: tensorflow-metadata in /usr/local/lib/python3.12/dist-packages (1.21.0)
Requirement already satisfied: absl-py<3.0.0,>=0.9 in /usr/local/lib/python3.12/dist-packages (from tensorflow-metadata) (1.
Requirement already satisfied: googleapis-common-protos<2,>=1.56.4 in /usr/local/lib/python3.12/dist-packages (from tensorflow
Downloading protobuf-7.35.1-cp310-abi3-manylinux2014_x86_64.whl (327 kB)
----- 327.1/327.1 kB 11.3 MB/s eta 0:00:00
Installing collected packages: protobuf
  Attempting uninstall: protobuf
    Found existing installation: protobuf 5.29.6
    Uninstalling protobuf-5.29.6:
      Successfully uninstalled protobuf-5.29.6
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
google-ai-generativelanguage 0.6.15 requires protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.
grpcio-status 1.71.2 requires protobuf<6.0dev,>=5.26.1, but you have protobuf 7.35.1 which is incompatible.
ydf 0.15.0 requires protobuf<7.0.0,>=5.29.1, but you have protobuf 7.35.1 which is incompatible.
Successfully installed protobuf-7.35.1
```

```
import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

# 1. Changement et préparation des données (Cats vs Dogs)

print("Chargement du dataset Cats vs Dogs depuis TFDS...")

# Découpage manuel demandé : 80% train, 10% val, 10% test
splits = ['train[:80%]', 'train[80%:90%]', 'train[90%:]']

(train_ds, val_ds, test_ds), ds_info = tfds.load(
    'cats_vs_dogs',
    split=splits,
    with_info=True,
    as_supervised=True # Pour obtenir directement (image, label)
)

IMG_SIZE = 224

def preprocess_image(image, label):
    # Redimensionnement des images en 224x224 selon les exigences du sujet
    image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))
    # Normalisation de base des pixels (0 à 1)
    image = tf.cast(image, tf.float32) / 255.0
    return image, label

# Application du prétraitement et mise en cache pour accélérer l'entraînement
BATCH_SIZE = 32

train_ds = train_ds.map(preprocess_image, num_parallel_calls=tf.data.AUTOTUNE).shuffle(1000).batch(BATCH_SIZE).prefetch(tf.
val_ds = val_ds.map(preprocess_image, num_parallel_calls=tf.data.AUTOTUNE).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
test_ds = test_ds.map(preprocess_image, num_parallel_calls=tf.data.AUTOTUNE).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

print("--- Vérification du découpage ---")
print(f"Nombre total d'images : {ds_info.splits['train'].num_examples}")
print("Données préparées avec succès en Train (80%), Validation (10%) et Test (10%) !")
```

```
Chargement du dataset Cats vs Dogs depuis TFDS...
WARNING:absl:Variant folder /root/tensorflow_datasets/cats_vs_dogs/4.0.1 has no dataset_info.json
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size, total: Unknown size) to /ro
DI Completed...: 100% 1/1 [00:12<00:00, 12.60s/ url]

DI Size...: 100% 786/786 [00:12<00:00, 73.80 MiB/s]
WARNING:absl:1738 images were corrupted and were skipped
Dataset cats_vs_dogs downloaded and prepared to /root/tensorflow_datasets/cats_vs_dogs/4.0.1. Subsequent calls will reuse th
--- Vérification du découpage ---
Nombre total d'images : 23262
Données préparées avec succès en Train (80%), Validation (10%) et Test (10%) !
```

```
# 2. Choix des architectures pour comparaison & 3. Entraînement
```

```

# Définition de la couche de Data Augmentation pour éviter le surapprentissage
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])

# Liste des architectures pré-entraînées à comparer
architectures = {
    "VGG16": tf.keras.applications.VGG16,
    "MobileNetV2": tf.keras.applications.MobileNetV2,
    "ResNet50": tf.keras.applications.ResNet50,
    "EfficientNetB0": tf.keras.applications.EfficientNetB0
}

# Dictionnaire pour sauvegarder les historiques d'entraînement de chaque modèle
trained_models = {}
histories = {}

# Boucle pour configurer et entraîner chaque modèle l'un après l'autre
for name, model_fn in architectures.items():
    print(f"\n" + "="*60)
    print(f"Configuration de l'architecture : {name}")
    print("="*60)

    # Importer le modèle pré-entraîné sur ImageNet sans sa tête d'origine
    base_model = model_fn(
        input_shape=(224, 224, 3),
        include_top=False,
        weights='imagenet'
    )

    # Geler les couches convolutionnelles pour utiliser les features déjà apprises
    base_model.trainable = False

    # Ajouter des couches fully connected pour la classification binaire
    model = models.Sequential([
        layers.Input(shape=(224, 224, 3)),
        data_augmentation,          # Application de la Data Augmentation
        base_model,                  # Le cœur convolutif gelé
        layers.GlobalAveragePooling2D(), # Vectorisation des features maps
        layers.Dense(128, activation='relu'), # Couche dense intermédiaire
        layers.Dropout(0.5),          # Régularisation
        layers.Dense(1, activation='sigmoid') # Sortie binaire (0 = Chat, 1 = Chien)
    ])

    # 3. Paramètres d'entraînement
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss='binary_crossentropy',      # Perte imposée pour le binaire
        metrics=['accuracy']
    )

    # Lancement de l'entraînement sur 3 époques (parfait pour comparer en temps limité)
    print(f"Début de l'entraînement pour {name}...")
    history = model.fit(
        train_ds,
        epochs=3,
        validation_data=val_ds
    )

    # Sauvegarde du modèle et de son historique pour la partie Évaluation
    trained_models[name] = model
    histories[name] = history
    print(f"Entraînement de {name} terminé avec succès !")

```

```

=====
Configuration de l'architecture : VGG16
=====
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16\_weights\_tf\_dim\_ordering\_tf\_kernels.h5
58889256/58889256 — 0s 0us/step
Début de l'entraînement pour VGG16...
Epoch 1/3
582/582 — 110s 172ms/step - accuracy: 0.8113 - loss: 0.4096 - val_accuracy: 0.9015 - val_loss: 0.2263
Epoch 2/3
582/582 — 106s 179ms/step - accuracy: 0.8724 - loss: 0.2972 - val_accuracy: 0.9140 - val_loss: 0.1998
Epoch 3/3
582/582 — 105s 178ms/step - accuracy: 0.8811 - loss: 0.2786 - val_accuracy: 0.9136 - val_loss: 0.2034
Entraînement de VGG16 terminé avec succès !

=====
Configuration de l'architecture : MobileNetV2
=====

```

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet\_v2/mobilenet\_v2\_weights\_tf\_dim\_ordering\_tf\_data\_format\_101\_135\_224/mobilenet\_v2\_weights\_tf\_dim\_ordering\_tf\_data\_format\_101\_135\_224.h5
9406464/9406464 ————— 0s 0us/step
Début de l'entraînement pour MobileNetV2...
Epoch 1/3
582/582 ————— 53s 79ms/step - accuracy: 0.9549 - loss: 0.1139 - val_accuracy: 0.9811 - val_loss: 0.0516
Epoch 2/3
582/582 ————— 45s 74ms/step - accuracy: 0.9649 - loss: 0.0889 - val_accuracy: 0.9845 - val_loss: 0.0417
Epoch 3/3
582/582 ————— 45s 75ms/step - accuracy: 0.9663 - loss: 0.0828 - val_accuracy: 0.9832 - val_loss: 0.0416
Entraînement de MobileNetV2 terminé avec succès !

=====
Configuration de l'architecture : ResNet50
=====
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50\_weights\_tf\_dim\_ordering\_tf\_data\_format\_101\_135\_224/resnet50\_weights\_tf\_dim\_ordering\_tf\_data\_format\_101\_135\_224.h5
94765736/94765736 ————— 1s 0us/step
Début de l'entraînement pour ResNet50...
Epoch 1/3
582/582 ————— 107s 168ms/step - accuracy: 0.5574 - loss: 0.6861 - val_accuracy: 0.5791 - val_loss: 0.6706
Epoch 2/3
582/582 ————— 96s 163ms/step - accuracy: 0.5792 - loss: 0.6763 - val_accuracy: 0.6010 - val_loss: 0.6632
Epoch 3/3
582/582 ————— 96s 162ms/step - accuracy: 0.5843 - loss: 0.6721 - val_accuracy: 0.6105 - val_loss: 0.6555
Entraînement de ResNet50 terminé avec succès !

=====
Configuration de l'architecture : EfficientNetB0
=====
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/efficientnetb0/efficientnetb0\_weights\_tf\_dim\_ordering\_tf\_data\_format\_101\_135\_224/efficientnetb0\_weights\_tf\_dim\_ordering\_tf\_data\_format\_101\_135\_224.h5
16705208/16705208 ————— 0s 0us/step
Début de l'entraînement pour EfficientNetB0...
Epoch 1/3
582/582 ————— 73s 103ms/step - accuracy: 0.4987 - loss: 0.6957 - val_accuracy: 0.5185 - val_loss: 0.6931
Epoch 2/3
582/582 ————— 58s 97ms/step - accuracy: 0.5011 - loss: 0.6932 - val_accuracy: 0.5185 - val_loss: 0.6931
Epoch 3/3
582/582 ————— 57s 96ms/step - accuracy: 0.5031 - loss: 0.6932 - val_accuracy: 0.4815 - val_loss: 0.6932
Entraînement de EfficientNetB0 terminé avec succès !

```

Start coding or [generate](#) with AI.

```

# 3. Entraînement du modèle (Configuration & Optimisation avec Callbacks)

# Définition de la Data Augmentation pour augmenter la diversité et éviter le surapprentissage
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])

# Liste des architectures pré-entraînées
architectures = {
    "VGG16": tf.keras.applications.VGG16,
    "MobileNetV2": tf.keras.applications.MobileNetV2,
    "ResNet50": tf.keras.applications.ResNet50,
    "EfficientNetB0": tf.keras.applications.EfficientNetB0
}

trained_models = {}
histories = {}

# Boucle pour configurer, compiler et entraîner chaque architecture
for name, model_fn in architectures.items():
    print(f"\n" + "="*60)
    print(f"Préparation et Entraînement de : {name}")
    print("="*60)

    # Importation du modèle de base (couches convolutives gelées)
    base_model = model_fn(input_shape=(224, 224, 3), include_top=False, weights='imagenet')
    base_model.trainable = False

    # Assemblage du modèle avec la Data Augmentation et le classifieur binaire
    model = models.Sequential([
        layers.Input(shape=(224, 224, 3)),
        data_augmentation,
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid') # Classification binaire (0=Chat, 1=Chien)
    ])

    # Définition de la perte (binary_crossentropy) et choix de l'optimiseur (Adam)
    model.compile(

```

```

optimizer=tf.keras.optimizers.Adam(learning_rate=0.001), # Learning rate adapté
loss='binary_crossentropy', # Fonction de perte binaire
metrics=['accuracy']
)

# Utilisation de Early Stopping et Checkpointing pour optimiser l'entraînement
callbacks_list = [
    # Arrête l'entraînement si la perte de validation ne s'améliore plus pendant 2 époques
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=2,
        restore_best_weights=True
    ),
    # Sauvegarde uniquement le meilleur modèle basé sur la précision de validation
    tf.keras.callbacks.ModelCheckpoint(
        filepath=f'best_model_{name}.keras',
        monitor='val_accuracy',
        save_best_only=True
    )
]

# Lancement de l'entraînement (fixé à 10 époques max, le Early Stopping arrêtera si nécessaire)
print(f"Entraînement de {name} en cours...")
history = model.fit(
    train_ds,
    epochs=10,
    validation_data=val_ds,
    callbacks=callbacks_list,
    verbose=1
)

# Sauvegarde des objets pour l'étape suivante (Évaluation)
trained_models[name] = model
histories[name] = history
print(f"Modèle {name} entraîné et optimisé !")

```

Préparation et Entraînement de : VGG16

=====

Entraînement de VGG16 en cours...

Epoch 1/10

582/582 ————— 109s 182ms/step - accuracy: 0.8115 - loss: 0.4127 - val_accuracy: 0.8994 - val_loss: 0.2454

Epoch 2/10

582/582 ————— 106s 179ms/step - accuracy: 0.8729 - loss: 0.2983 - val_accuracy: 0.9119 - val_loss: 0.2039

Epoch 3/10

582/582 ————— 106s 179ms/step - accuracy: 0.8780 - loss: 0.2783 - val_accuracy: 0.9175 - val_loss: 0.1899

Epoch 4/10

582/582 ————— 106s 179ms/step - accuracy: 0.8833 - loss: 0.2668 - val_accuracy: 0.9187 - val_loss: 0.1937

Epoch 5/10

582/582 ————— 106s 179ms/step - accuracy: 0.8896 - loss: 0.2575 - val_accuracy: 0.9226 - val_loss: 0.1811

Epoch 6/10

582/582 ————— 106s 179ms/step - accuracy: 0.8889 - loss: 0.2552 - val_accuracy: 0.9205 - val_loss: 0.1906

Epoch 7/10

582/582 ————— 105s 179ms/step - accuracy: 0.8881 - loss: 0.2562 - val_accuracy: 0.9179 - val_loss: 0.1885

Modèle VGG16 entraîné et optimisé !

=====

Préparation et Entraînement de : MobileNetV2

=====

Entraînement de MobileNetV2 en cours...

Epoch 1/10

582/582 ————— 52s 79ms/step - accuracy: 0.9548 - loss: 0.1156 - val_accuracy: 0.9824 - val_loss: 0.0504

Epoch 2/10

582/582 ————— 45s 75ms/step - accuracy: 0.9649 - loss: 0.0892 - val_accuracy: 0.9819 - val_loss: 0.0446

Epoch 3/10

582/582 ————— 45s 75ms/step - accuracy: 0.9689 - loss: 0.0778 - val_accuracy: 0.9789 - val_loss: 0.0610

Epoch 4/10

582/582 ————— 45s 76ms/step - accuracy: 0.9689 - loss: 0.0775 - val_accuracy: 0.9819 - val_loss: 0.0459

Modèle MobileNetV2 entraîné et optimisé !

=====

Préparation et Entraînement de : ResNet50

=====

Entraînement de ResNet50 en cours...

Epoch 1/10

582/582 ————— 107s 169ms/step - accuracy: 0.5490 - loss: 0.6902 - val_accuracy: 0.5189 - val_loss: 0.6882

Epoch 2/10

582/582 ————— 98s 165ms/step - accuracy: 0.5668 - loss: 0.6781 - val_accuracy: 0.6071 - val_loss: 0.6651

Epoch 3/10

582/582 ————— 97s 163ms/step - accuracy: 0.5760 - loss: 0.6746 - val_accuracy: 0.6023 - val_loss: 0.6698

Epoch 4/10

582/582 ————— 97s 165ms/step - accuracy: 0.5778 - loss: 0.6737 - val_accuracy: 0.6178 - val_loss: 0.6703

Modèle ResNet50 entraîné et optimisé !

=====

Préparation et Entraînement de : EfficientNetB0

```
Epoch 1/10
582/582 ————— 71s 102ms/step - accuracy: 0.5023 - loss: 0.6954 - val_accuracy: 0.5185 - val_loss: 0.6931
Epoch 2/10
582/582 ————— 57s 95ms/step - accuracy: 0.5027 - loss: 0.6932 - val_accuracy: 0.4815 - val_loss: 0.6932
Epoch 3/10
582/582 ————— 57s 96ms/step - accuracy: 0.5015 - loss: 0.6932 - val_accuracy: 0.4815 - val_loss: 0.6932
Modèle EfficientNetB0 entraîné et optimisé !
```

```
# =====
# 4. Évaluation (Calcul des métriques complexes et Courbes ROC)
# =====

import numpy as np
import pandas as pd
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score, roc_curve

print("Extraction des étiquettes réelles (y_true) du jeu de test...")
# Extraction des vrais labels pour les calculs de Scikit-Learn
y_true = np.concatenate([y for x, y in test_ds], axis=0)

results_summary = []
roc_curves_data = {}

# Boucle d'évaluation sur les modèles précédemment entraînés et sauvegardés
for name in architectures.keys():
    print(f"\n Évaluation détaillée du modèle : {name}...")
    model = trained_models[name]

    # 1. Prédiction des probabilités continues (nécessaires pour l'AUC et la courbe ROC)
    y_pred_probs = model.predict(test_ds, verbose=0).ravel()

    # 2. Conversion en classes binaires strictes (seuil standard à 0.5)
    y_pred_classes = (y_pred_probs > 0.5).astype(int)

    # 3. Calcul des métriques demandées par l'énoncé
    loss, acc = model.evaluate(test_ds, verbose=0)
    precision = precision_score(y_true, y_pred_classes)
    recall = recall_score(y_true, y_pred_classes)
    f1 = f1_score(y_true, y_pred_classes)
    auc_score = roc_auc_score(y_true, y_pred_probs)

    # 4. Sauvegarde des données ROC (False Positive Rate et True Positive Rate)
    fpr, tpr, _ = roc_curve(y_true, y_pred_probs)
    roc_curves_data[name] = (fpr, tpr, auc_score)

    # 5. Stockage dans la liste pour le tableau récapitulatif
    results_summary.append({
        "Modèle / Architecture": name,
        "Accuracy (%)": round(acc * 100, 2),
        "Précision (%)": round(precision * 100, 2),
        "Rappel (%)": round(recall * 100, 2),
        "F1-Score (%)": round(f1 * 100, 2),
        "AUC Score": round(auc_score, 4)
    })

# --- AFFICHAGE DU TABLEAU COMPARATIF FINAL ---
print("\n" + "="*70)
print(" TABLEAU COMPARATIF DES PERFORMANCES DU TRANSFER LEARNING")
print("="*70)
df_comparatif = pd.DataFrame(results_summary)
display(df_comparatif)

# --- AFFICHAGE DES COURBES ROC SUPERPOSÉES ---
plt.figure(figsize=(10, 7))
for name, (fpr, tpr, auc_score) in roc_curves_data.items():
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc_score:.4f})', lw=2)

plt.plot([0, 1], [0, 1], 'k--', label='Aléatoire (AUC = 0.5000)', lw=1.5)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('Taux de faux positifs (FPR)')
plt.ylabel('Taux de vrais positifs (TPR)')
plt.title('Comparaison des courbes ROC - Transfer Learning (Cats vs Dogs)')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Extraction des étiquettes réelles (y_true) du jeu de test...

Évaluation détaillée du modèle : VGG16...

Évaluation détaillée du modèle : MobileNetV2...

Évaluation détaillée du modèle : ResNet50...

Évaluation détaillée du modèle : EfficientNetB0...

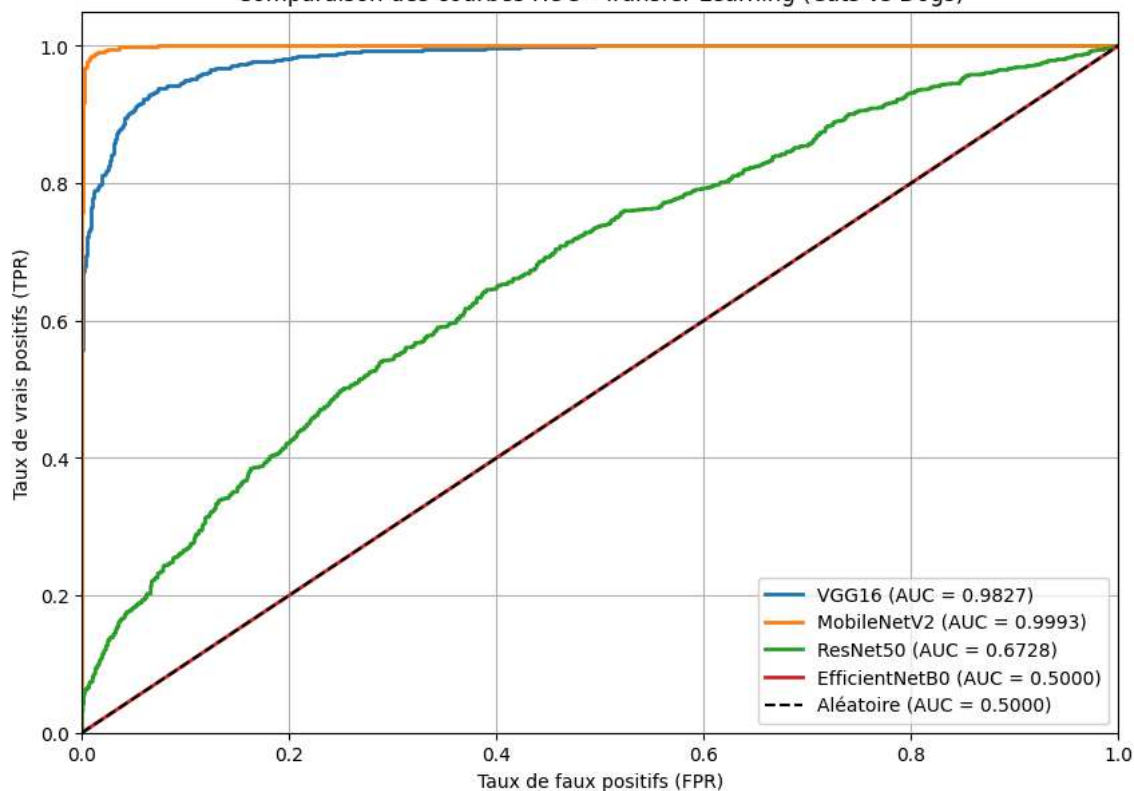
=====

TABLEAU COMPARATIF DES PERFORMANCES DU TRANSFER LEARNING

=====

	Modèle / Architecture	Accuracy (%)	Précision (%)	Rappel (%)	F1-Score (%)	AUC Score	
0	VGG16	92.95	94.02	91.77	92.88	0.9827	
1	MobileNetV2	98.71	98.46	98.97	98.72	0.9993	
2	ResNet50	62.12	60.64	69.64	64.83	0.6728	
3	EfficientNetB0	50.13	50.13	100.00	66.78	0.5000	

Comparaison des courbes ROC - Transfer Learning (Cats vs Dogs)



Next steps: [Generate code with df_comparatif](#) [New interactive sheet](#)

```
# =====
# 5. Analyse : Visualisation des Feature Maps (Modèle MobileNetV2)
# =====

# 1. Récupérer un batch d'images de test pour l'extraction
images, labels = next(iter(test_ds))
sample_image = images[0] # On prend la première image du batch

# 2. Créer un modèle intermédiaire qui s'arrête à la première couche convolutive du base_model
# MobileNetV2 possède des sous-couches imbriquées, on extrait la première couche de type Conv2D
base_layers = trained_models["MobileNetV2"].get_layer("mobilenetv2_1.00_224").layers
first_conv_layer = [l for l in base_layers if "conv" in l.name][0]

activation_model = tf.keras.models.Model(
    inputs=trained_models["MobileNetV2"].get_layer("mobilenetv2_1.00_224").input,
    outputs=first_conv_layer.output
)

# 3. Prédire pour obtenir les feature maps (on ajoute une dimension de batch)
feature_maps = activation_model.predict(tf.expand_dims(sample_image, axis=0))

# 4. Affichage de l'image originale et de ses 4 premières feature maps extractrices
plt.figure(figsize=(12, 6))

# Image d'origine
```

```
plt.subplot(1, 5, 1)
plt.imshow(sample_image)
plt.title("Image Originale")
plt.axis("off")

# Les 4 premières cartes de caractéristiques (Filtres de détection de contours)
for i in range(4):
    plt.subplot(1, 5, i + 2)
    plt.imshow(feature_maps[0, :, :, i], cmap="viridis")
    plt.title(f"Feature Map {i+1}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

1/1 ————— 2s 2s/step

